
Delving into Muon and Beyond: Deep Analysis and Extensions

Xianbiao Qi^{*1} Marco Chen^{*2} Jiaquan Ye¹ Yelin He¹ Rong Xiao^{†1}

Abstract

The Muon optimizer has recently attracted considerable attention for its strong empirical performance and use of orthogonalized updates on matrix-shaped parameters, yet its underlying mechanisms and relationship to adaptive optimizers such as Adam remain insufficiently understood. In this work, we aim to address these questions through a unified spectral perspective. Specifically, we view Muon as the $p = 0$ endpoint of a family of spectral transformations of the form $U\Sigma^pV^\top$, and consider additional variants with $p = \frac{1}{2}$, $p = \frac{1}{4}$, and $p = 1$. These transformations are applied to both first-moment updates, as in momentum SGD, and to root-mean-square (RMS) normalized gradient updates as in Adam. To enable efficient computation, we develop a coupled Newton iteration that avoids explicit singular value decomposition. Across controlled experiments, we find that RMS-normalized updates yield more stable optimization than first-moment updates. Moreover, while spectral compression provides strong stabilization benefits under first-moment updates, the Muon update ($p = 0$) does not consistently outperform Adam. These results suggest that Muon is best understood as an effective form of spectral normalization, but not a universally superior optimization method. Our code is available at <https://github.com/Ocram7/BeyondMuon>.

1. Introduction

Optimization methods (Robbins & Monro, 1951; Nesterov, 1983; Duchi et al., 2011; Hinton et al., 2012; Kingma & Ba, 2014; Loshchilov & Hutter, 2019; Gupta et al., 2018; Jordan et al., 2024) are a cornerstone of modern deep learning.

^{*}Equal contribution ¹Intellifusion Inc. ²Tsinghua University, marco.ty.chen@gmail.com. Correspondence to: Rong Xiao <rongxiao@gmail.com>.

Among them, Adam (Kingma & Ba, 2014) and its variants (Loshchilov & Hutter, 2019; Shazeer & Stern, 2018) are widely adopted due to their strong empirical stability and robustness across architectures and scales. By combining momentum with variance-based normalization, Adam incorporates second-moment information while remaining computationally efficient, and has consequently become the default optimizer in large-scale neural network training (Touvron et al., 2023a;b; Dubey et al., 2024; Guo et al., 2025; Yang et al., 2025).

Recently, Muon (Jordan et al., 2024) has attracted growing attention as an alternative optimization strategy, particularly in large language models (Team et al., 2025). Unlike most conventional optimizers that operate element-wise, Muon applies matrix-level transformations to gradients, enforcing orthogonality through a spectral operation. Empirical evidence suggests that Muon can improve convergence behavior, leading to growing interest in the community. Despite growing adoption, the effectiveness and role of Muon remain insufficiently understood. Existing work has either focused on empirical performance gains (Jordan et al., 2024; Team et al., 2025) or on mathematical analysis (Newhouse, 2025; Bernstein, 2025a; Buchanan, 2025), but these studies are often accompanied by additional techniques such as QK-Norm or QK-Clip (Team et al., 2025). Consequently, it remains unclear to what extent the reported improvements can be attributed to Muon itself, how Muon relates to established adaptive optimizers such as Adam, and whether Muon can be systematically improved. These ambiguities motivate a careful and controlled re-examination of Muon.

In this work, our goal is to study the effectiveness of the Muon optimizer through a unified spectral framework and controlled empirical evaluation. We view Muon as the $p = 0$ endpoint of a family of spectral transformations of the form $U\Sigma^pV^\top$, and consider fractional variants with $p = \frac{1}{2}$ and $p = \frac{1}{4}$, as well as $p = 1$, which recovers a standard gradient update. We evaluate these transformations under both first-moment updates, as in momentum SGD, and root-mean-square-normalized updates, as in Adam. To make fractional spectral updates practical, we introduce a coupled Newton-Schulz iteration method that computes $U\Sigma^{\frac{1}{2}}V^\top$ and $U\Sigma^{\frac{1}{4}}V^\top$ using only matrix multiplications, avoiding explicit SVD.

To ensure fair and interpretable comparisons, we design our experiments to be highly controlled. We explicitly decouple matrix and vector learning rates for all methods, and avoid weight decay as well as auxiliary stabilizing techniques like QK-Norm and QK-Clip in the main experiments. Later, we ablate weight decay and QK-Norm to examine whether the trends from our primary experiments persist under more practical settings.

Under these settings, our experiments yield three key findings: (i) Muon ($p = 0$) is crucial for stabilizing first-moment updates and substantially improves robustness over mSGD; (ii) when applied to RMS-normalized updates, Muon-like orthogonalization does not consistently outperform Adam and is matched or exceeded by partial spectral compression; and (iii) across spectral variants, RMS-normalized variants are generally stronger than their first-moment counterparts.

Our main contributions are summarized as follows:

- We introduce a unified spectral framework $U\Sigma^pV^\top$ that places Muon as the $p = 0$ endpoint and propose intermediate variants with $p = \frac{1}{2}$ and $p = \frac{1}{4}$.
- We develop a coupled Newton–Schulz iteration method that enables efficient computation of the fractional spectral updates $p = \frac{1}{2}$ and $p = \frac{1}{4}$ without explicit SVD, and analyze its approximation quality and runtime overhead.
- We provide a controlled empirical comparison between Muon, Adam, and their spectral variants across first-moment and RMS-normalized updates, isolating spectral effects from other confounding techniques.

Conflict of Interest Disclosure. The authors declare no financial interests beyond the disclosed affiliations.

2. Related Work

Optimizers before Transformer. Before the Transformer (Vaswani et al., 2017), CNNs (LeCun et al., 2002; He et al., 2016) and shallow recurrent models such as LSTMs (Hochreiter & Schmidhuber, 1997) dominated neural network design. These architectures were typically of moderate depth with limited cumulative nonlinearity, resulting in relatively small variation in layer-wise Lipschitz constants (Qi et al., 2023a;b). Consequently, training with a global learning rate was generally effective. Under this regime, classical stochastic optimization methods (Bottou et al., 2018) were sufficient. Stochastic Gradient Descent (SGD) (Robbins & Monro, 1951) and its variants (Nesterov, 1983; Johnson & Zhang, 2013), often combined with simple learning rate schedules, were the standard choices.

Although adaptive methods such as Adagrad (Duchi et al., 2011), RMSProp (Hinton et al., 2012), and Adam (Kingma

& Ba, 2014) had been proposed, they were not widely adopted at the time. This can be attributed to two main factors. First, weight decay was primarily viewed as a regularization technique, and its interaction with adaptive optimizers was poorly understood; in particular, coupling weight decay with gradients rather than weights significantly degraded the performance of Adam-type methods (Loshchilov & Hutter, 2019). Second, in CNN-based models, inter-layer differences in Lipschitz constants were insufficient to necessitate per-parameter adaptivity, and momentum SGD already achieved strong empirical performance. As a result, Adam did not consistently outperform mSGD in this setting.

Optimizers after Transformer. The introduction of Transformers (Vaswani et al., 2017) fundamentally altered the optimization landscape of deep learning. Compared to earlier architectures, Transformer-based models are substantially deeper and exhibit stronger nonlinearities (Shazeer, 2020), arising from attention mechanisms (Vaswani et al., 2017), residual connections (He et al., 2016), and normalization layers (Ba et al., 2016). Consequently, different layers and parameter groups often exhibit markedly different curvature and gradient statistics, often leading to training instability and suboptimal convergence.

AdamW (Loshchilov & Hutter, 2019) resolved the improper coupling between weight decay and adaptive learning rates in Adam, leading to significantly improved performance on Transformer models. As a result, AdamW consistently outperforms momentum SGD in large-scale Transformer training and has become a standard optimizer choice. Subsequent work has proposed a range of alternative optimizers, including Adafactor (Shazeer & Stern, 2018), SignSGD (Bernstein et al., 2018), LAMB (You et al., 2019), Adan (Xie et al., 2024), Lion (Chen et al., 2023), Sophia (Liu et al., 2024), Mars (Yuan et al., 2025), and related variants (Zhang et al., 2025; Liang et al., 2025). These methods primarily target improved stability, efficiency, or convergence in large-scale, highly nonlinear training regimes.

Matrix-based Optimizers. Matrix-based optimizers exploit the structured geometry of parameters by operating directly on matrix-valued variables, rather than treating parameters as independent scalars. Early work in this direction focused on making second-order optimization tractable via structured approximations. K-FAC (Martens & Grosse, 2015) introduced a Kronecker-factored approximation of the Fisher information matrix, enabling efficient layer-wise preconditioning, while Shampoo (Gupta et al., 2018) extended this idea by applying Kronecker factorizations along multiple matrix dimensions, leading to improved stability. These methods share the goal of incorporating curvature information through structured matrix approximations.

More recently, Muon (Jordan et al., 2024) has emerged as a matrix-based optimizer that departs from explicit cur-

vature modeling. Muon directly updates matrix-valued parameters via orthogonalization and spectral transformations computed using Newton–Schulz iterations (Schulz, 1933; Higham, 2008), relying solely on matrix multiplications. Since its introduction, Muon has inspired a growing body of follow-up work (Team et al., 2025; Si et al., 2025; Frans et al., 2025; Su, 2025; Lau et al., 2025), including analyses that clarify its numerical linear algebra foundations (Bernstein, 2025a; Newhouse, 2025) and geometric interpretations that view it as implicit manifold-aware optimization (Bernstein, 2025b; Buchanan, 2025).

Remarks. Unlike several previous works on optimizer benchmarking (Wen et al., 2025; Frans et al., 2025), our study is characterized by the following features: (1) we introduce a unified spectral perspective $U\Sigma^pV^\top$ with $p \in [0, 1]$, and instantiate it with four concrete choices, $p \in \{1, \frac{1}{2}, \frac{1}{4}, 0\}$. For $U\Sigma^{\frac{1}{2}}V^\top$ and $U\Sigma^{\frac{1}{4}}V^\top$, we develop a coupled Newton–Schulz algorithm to solve them. (2) we experimentally show that matrix-based optimizers with second-moment information are stronger than their pure first-moment counterparts. (3) we design our experiments to be highly controlled. We explicitly decouple matrix and vector learning rates for all methods, disable weight decay, and avoid auxiliary techniques. This allows us to isolate the intrinsic capabilities of each optimizer, enabling a fair and transparent comparison.

3. Delving into Muon and Beyond

3.1. Baselines: Adam and Muon

As in Muon, our study focuses exclusively on *matrix-shaped parameters* $W \in \mathbb{R}^{m \times n}$; all vector parameters are optimized using standard Adam. In this subsection, we briefly review Adam and Muon and fix notation.

Adam. Adam (Kingma & Ba, 2014) is a popular stochastic optimizer that combines momentum with adaptive, second-moment-based normalization. Given the gradient matrix $G_t = \nabla_{W_t} \ell(W_t)$ at iteration t , Adam maintains exponentially decaying estimates of the first and second moments:

$$M_t = \beta_1 M_{t-1} + (1 - \beta_1) G_t, \quad (1)$$

$$V_t = \beta_2 V_{t-1} + (1 - \beta_2) G_t^2, \quad (2)$$

and updates parameters via

$$W_{t+1} = W_t - \eta_t M_t \oslash \sqrt{V_t}. \quad (3)$$

Here, β_1 and β_2 are the first- and second-moment decay rates, η_t is the learning rate, and $\oslash$ denotes elementwise division. From a preconditioning perspective, Adam applies a diagonal matrix $V_t^{-1/2}$ to the momentum M_t , normalizing each coordinate by an estimate of its second-moment

and yielding bounded, scale-adaptive updates. For brevity, throughout the rest of this work we refer to $M_t \oslash \sqrt{V_t}$ as the *RMS-normalized update*, since V_t tracks an exponential moving average of squared gradients.

Muon. Muon (Jordan et al., 2024) replaces Adam’s elementwise normalization with a *spectral normalization* of the update matrix. Given the momentum matrix M_t with singular value decomposition $M_t = U\Sigma V^\top$, Muon defines the update direction as the polar factor

$$\mathcal{P}(M_t) := UV^\top,$$

which discards the singular values and retains only the left and right singular vectors. Computing the polar factor is typically referred to as matrix orthogonalization.

From an optimization perspective, orthogonalization enforces *unit magnitude along every singular direction*. In contrast to Adam, which normalizes updates on an elementwise basis via a diagonal preconditioner, Muon equalizes the strength of updates across all directions of the gradient matrix. As a result, Muon removes anisotropic scaling in the spectrum of M_t , yielding directionally balanced but magnitude-agnostic updates.

Newton–Schulz approximation. Computing the polar factor $\mathcal{P}(M_t) = UV^\top = M_t(M_t^\top M_t)^{-1/2}$ via an explicit SVD is prohibitively expensive for large models. However, we can approximate the inverse square root using *Newton–Schulz iteration*, which relies only on matrix multiplications. Let $A = \alpha M_t^\top M_t$, where the scaling constant α is chosen such that $\|I - A\|_2 \leq 1$. The iteration

$$Z_{k+1} = \frac{1}{2} Z_k (3I - AZ_k^2), \quad Z_0 = I, \quad (4)$$

converges quadratically to $\frac{1}{\sqrt{\alpha}}(M_t^\top M_t)^{-1/2}$. After K iterations, the polar factor is approximated as

$$\mathcal{P}(M_t) \approx \sqrt{\alpha} M_t Z_K. \quad (5)$$

In practice, Muon (Jordan et al., 2024) employs Algorithm 2 in Appendix B, which is a slightly more direct and aggressive version of the Newton–Schulz iteration shown in Equation 4. The official implementation directly computes the polar factor with a procedure that converges for initializations where the singular values are adequately normalized.

3.2. Momentum vs. RMS-Normalized Updates

Where to apply spectral transforms. Muon (Jordan et al., 2024) applies its spectral operation to the *first-moment* momentum M_t . However, in large-scale Transformer training, Adam-style optimizers that incorporate second-moment statistics generally outperform momentum SGD-based optimizers that only leverage first-moment information. This

observation motivates us to examine whether spectral transformations yield additional benefits when applied to rms-normalized updates. Concretely, we study the two inputs

$$\mathbf{O}_t^{\text{mom}} := \mathbf{M}_t \quad \text{and} \quad \mathbf{O}_t^{\text{rms}} := \mathbf{M}_t \oslash \sqrt{\mathbf{V}_t},$$

which correspond to the first-moment update shown in Equation 1 and the rms-normalized update from Equation 3.

How $\mathbf{O}_t^{\text{mom}}$ and $\mathbf{O}_t^{\text{rms}}$ differ. The key distinction between these two regimes lies in how update magnitudes are controlled. In first-order momentum methods, the update $\mathbf{M}_t$ aggregates gradients over time without explicit normalization by their scale. As a result, the norm of the update can grow with accumulated gradient magnitude, especially under high variance or anisotropic curvature. Consequently, the update scale is not bounded by the optimizer.

In contrast, RMS-normalized updates explicitly rescale the momentum using second-moment statistics. The update $\mathbf{M}_t \oslash \sqrt{\mathbf{V}_t}$ normalizes each coordinate by an estimate of its raw second moment, yielding an update whose magnitude is provably bounded. For Adam-style methods, this bound, $\frac{1-\beta_1}{\sqrt{1-\beta_2}}$ (Qi et al., 2023b), depends only on the decay rates and is independent of the raw gradient magnitude. This intrinsic normalization leads to improved numerical stability and more predictable optimization dynamics.

3.3. A Spectral Family of Transformations

We introduce a family of spectral gradient transformations that enables a unified analysis of Muon-inspired spectral methods. Given the SVD of $\mathbf{O}_t$ as $\mathbf{O}_t = \mathbf{U}\Sigma\mathbf{V}^\top$, we define the general spectral transformation

$$\Psi_p(\mathbf{O}_t) := \mathbf{U}\Sigma^p\mathbf{V}^\top, \quad p \in [0, 1]. \quad (6)$$

The exponent p directly controls how the singular spectrum is rescaled. Writing the SVD in its rank-one form, we obtain

$$\Psi_p(\mathbf{O}_t) = \sum_{i=1}^d \sigma_i^p \mathbf{u}_i \mathbf{v}_i^\top, \quad (7)$$

which makes it explicit that decreasing p progressively compresses the singular values $\{\sigma_i\}$ while preserving the singular directions $\{\mathbf{u}_i, \mathbf{v}_i\}$.

This formulation subsumes several existing methods and enables systematic comparisons across spectral behaviors. In particular, $p = 1$ leaves the input unchanged, recovering standard methods like mSGD and Adam, while $p = 0$ maps all nonzero singular values to 1, yielding an orthogonalized update direction. When the input is the first-moment momentum $\mathbf{O}_t = \mathbf{O}_t^{\text{mom}}$, $p = 0$ corresponds to Muon.

3.4. Instantiating the Spectral Family

From a spectral operator to concrete optimizers. We instantiate our spectral update family by applying the map

$\Psi_p(\cdot)$ from Equation 6 to one of two matrix-valued inputs: the first-moment momentum $\mathbf{O}_t^{\text{mom}}$, or the RMS-normalized update $\mathbf{O}_t^{\text{rms}}$. We consider four spectral exponents

$$p \in \left\{1, \frac{1}{2}, \frac{1}{4}, 0\right\},$$

where $p = 1$ leaves the input unchanged and $p = 0$ collapses the spectrum to the polar factor. The intermediate powers $p = \frac{1}{2}$ and $p = \frac{1}{4}$ provide milder alternatives to Muon’s fully flattened spectrum and can be computed efficiently via coupled Newton–Schulz iteration, as described in Subsection 3.5. This yields the controlled interpolation

$$\mathbf{U}\Sigma^1\mathbf{V}^\top \rightarrow \mathbf{U}\Sigma^{1/2}\mathbf{V}^\top \rightarrow \mathbf{U}\Sigma^{1/4}\mathbf{V}^\top \rightarrow \mathbf{U}\Sigma^0\mathbf{V}^\top.$$

Combining two inputs with four exponents produces the eight optimizer instances evaluated in this work.

Naming convention. Each instance is identified by its *input family* and *spectral exponent*. We write

$$\text{BASEX} : \Delta \mathbf{W}_t \propto \Psi_p(\mathbf{O}_t),$$

where $\text{BASE} \in \{\text{mSGD}, \text{Adam}\}$ selects $\mathbf{O}_t^{\text{mom}}$ or $\mathbf{O}_t^{\text{rms}}$, and the suffix $\text{X} \in \{\emptyset, \text{S}, \text{Q}, \text{Z}\}$ encodes the exponent

$$\begin{aligned} \emptyset &\leftrightarrow p = 1 \text{ (identity)}, & \text{S} &\leftrightarrow p = \frac{1}{2} \text{ (square-root)}, \\ \text{Q} &\leftrightarrow p = \frac{1}{4} \text{ (quarter-power)}, & \text{Z} &\leftrightarrow p = 0 \text{ (zero-power)}. \end{aligned}$$

Momentum-input family ($\mathbf{O}_t^{\text{mom}} = \mathbf{M}_t$). Under the momentum input, the four instances are

$$\text{mSGD}, \text{mSGDS}, \text{mSGDQ}, \text{mSGDZ},$$

corresponding to $p \in \{1, \frac{1}{2}, \frac{1}{4}, 0\}$ respectively. The end-point **mSGDZ** is equivalent to **Muon** since $\Psi_0(\mathbf{M}_t) = \mathcal{P}(\mathbf{M}_t) = \mathbf{U}\mathbf{V}^\top$.

RMS-normalized-input family ($\mathbf{O}_t^{\text{rms}} = \mathbf{M}_t \oslash \sqrt{\mathbf{V}_t}$). Under the RMS-normalized input, the four instances are

$$\text{Adam}, \text{AdamS}, \text{AdamQ}, \text{AdamZ},$$

again corresponding to $p \in \{1, \frac{1}{2}, \frac{1}{4}, 0\}$ respectively.

Spectral anisotropy control. The four spectral variants in each input family share the same singular vectors ($\mathbf{U}, \mathbf{V}$) and differ only in how the singular values are rescaled. Thus, the effect of the spectral transformation can be understood as simply reshaping the anisotropy of the singular spectrum. To quantify anisotropy, we use the spectral condition number over the nonzero singular spectrum. Given the largest and smallest nonzero singular values of $\mathbf{O}_t$, denoted by $\sigma_{\max}$ and $\sigma_{\min}$, respectively, we define the condition number as

$$\kappa(\mathbf{O}_t) = \frac{\sigma_{\max}}{\sigma_{\min}}.$$

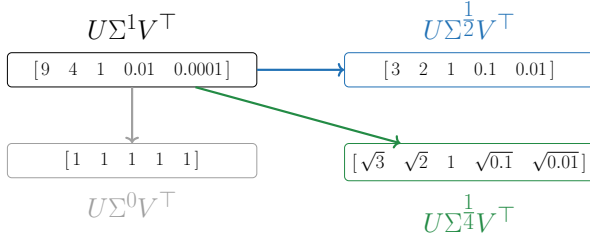


Figure 1. Effect of spectral exponent p on the singular spectrum (illustrative numbers). Decreasing p compresses the spectrum: large singular values are damped relative to small ones, and $p = 0$ maps all nonzero singular values to 1.

Intuitively, $\kappa(\mathbf{O}_t)$ measures the spread of the singular values. If the input is highly ill-conditioned and a small number of dominant singular directions disproportionately influence the update, $\kappa(\mathbf{O}_t)$ is large.

If the input has singular values $\{\sigma_i\}_{i=1}^d$, then Ψ_p produces singular values $\{\sigma_i^p\}_{i=1}^d$, so the condition number becomes

$$\kappa_p = \frac{\sigma_{\max}^p}{\sigma_{\min}^p} = \kappa(\mathbf{O}_t)^p.$$

Therefore, decreasing p reduces spectral anisotropy:

$$\kappa_1 \geq \kappa_{1/2} \geq \kappa_{1/4} \geq \kappa_0 = 1,$$

Indeed, for the toy spectrum in Figure 1, $\kappa_1 = 90000$, $\kappa_{1/2} = 300$, $\kappa_{1/4} = \sqrt{300}$, and $\kappa_0 = 1$. This viewpoint clarifies how the exponent p continuously interpolates between the original spectrum ($p = 1$) and complete spectral flattening ($p = 0$).

The effect of spectral compression. As shown in Figure 1, decreasing p progressively compresses the *entire* singular-value spectrum. Hence, directions associated with large singular values are attenuated, while those associated with small singular values (less than one) are amplified.

3.5. Efficient Computation

Computing $\Psi_{1/2}(\mathbf{O}_t)$ and $\Psi_{1/4}(\mathbf{O}_t)$. A practical challenge in spectral methods is computational efficiency, as direct SVD computation is infeasible for large models. Fortunately, we can rewrite $\Psi_{1/2}(\mathbf{O}_t)$ in a form that avoids explicit SVD. Let $\mathbf{O}_t = \mathbf{U}\Sigma\mathbf{V}^\top \in \mathbb{R}^{m \times n}$. Then,

$$\Psi_{1/2}(\mathbf{O}_t) = \mathbf{U}\Sigma^{1/2}\mathbf{V}^\top = \mathbf{O}_t(\mathbf{O}_t^\top\mathbf{O}_t)^{-1/4}.$$

Thus, it suffices to compute an inverse fourth root of the symmetric matrix $\mathbf{X} := \mathbf{O}_t^\top\mathbf{O}_t \in \mathbb{R}^{n \times n}$. We can compute $\mathbf{X}^{1/2}$ and $\mathbf{X}^{-1/2}$ efficiently using the coupled Newton-Schulz algorithm discussed below, and obtain $\mathbf{X}^{-1/4}$ by applying the same procedure to $\mathbf{X}^{1/2}$ (i.e., computing $(\mathbf{X}^{1/2})^{-1/2} = \mathbf{X}^{-1/4}$). Consequently, $\Psi_{1/2}(\mathbf{O})$ can be implemented using only matrix multiplications as $\Psi_{1/2}(\mathbf{O}) = \mathbf{O}\mathbf{X}^{-1/4}$.

Algorithm 1 Coupled Newton-Schulz for $\mathbf{X}^{1/2}$ and $\mathbf{X}^{-1/2}$

Require: $\mathbf{X}$, number of iterations K

Ensure: $\mathbf{X}^{1/2}$, $\mathbf{X}^{-1/2}$

- 1: $\mathbf{Y}_0 \leftarrow \mathbf{X}$, $\mathbf{Z}_0 \leftarrow \mathbf{I}$
 - 2: $\alpha \leftarrow \|\mathbf{X}\|_F$
 - 3: $\mathbf{Y}_0 \leftarrow \mathbf{Y}_0/\alpha$, $k \leftarrow 0$
 - 4: **while** $k < K$ **do**
 - 5: $\mathbf{T}_k \leftarrow 3\mathbf{I} - \mathbf{Z}_k\mathbf{Y}_k$
 - 6: $\mathbf{Y}_{k+1} \leftarrow \frac{1}{2}\mathbf{Y}_k\mathbf{T}_k$
 - 7: $\mathbf{Z}_{k+1} \leftarrow \frac{1}{2}\mathbf{T}_k\mathbf{Z}_k$
 - 8: $k \leftarrow k + 1$
 - 9: **end while**
 - 10: **return** $\sqrt{\alpha}\mathbf{Y}_K$, $\frac{1}{\sqrt{\alpha}}\mathbf{Z}_K$
-

A similar matrix-multiplication-only procedure can be used for the quarter-power spectral transformations.

In practice, since

$$\Psi_{1/2}(\mathbf{O}_t) = \mathbf{O}_t(\mathbf{O}_t^\top\mathbf{O}_t)^{-1/4} = (\mathbf{O}_t\mathbf{O}_t^\top)^{-1/4}\mathbf{O}_t.$$

we can choose $\mathbf{X} = \mathbf{O}_t^\top\mathbf{O}_t \in \mathbb{R}^{n \times n}$ or $\mathbf{X} = \mathbf{O}_t\mathbf{O}_t^\top \in \mathbb{R}^{m \times m}$ depending on whether $m \geq n$ to reduce the computational overhead of the Newton-Schulz iteration. The same applies in the computation of $\Psi_{1/4}$.

Coupled Newton-Schulz Iteration. Coupled Newton-Schulz iteration (Higham, 2008) provides an efficient and numerically stable procedure for simultaneously computing the matrix square root $\mathbf{X}^{1/2}$ and its inverse $\mathbf{X}^{-1/2}$.

Starting from the initialization $\mathbf{Y}_0 = \mathbf{X}$ and $\mathbf{Z}_0 = \mathbf{I}$, the method applies a coupled update that repeatedly refines both quantities using only matrix multiplications. Specifically, each iteration is defined as

$$\begin{aligned} \mathbf{Y}_{k+1} &= \frac{1}{2}\mathbf{Y}_k(3\mathbf{I} - \mathbf{Z}_k\mathbf{Y}_k), \\ \mathbf{Z}_{k+1} &= \frac{1}{2}(3\mathbf{I} - \mathbf{Z}_k\mathbf{Y}_k)\mathbf{Z}_k, \end{aligned} \tag{8}$$

which symmetrically updates $\mathbf{Y}_k$ and $\mathbf{Z}_k$ through the shared correction term $3\mathbf{I} - \mathbf{Z}_k\mathbf{Y}_k$. When appropriately scaled, this coupled iteration converges quadratically, driving $\mathbf{Y}_k \rightarrow \mathbf{X}^{1/2}$ and $\mathbf{Z}_k \rightarrow \mathbf{X}^{-1/2}$ simultaneously.

To ensure numerical stability, a normalization step based on the Frobenius norm of $\mathbf{X}$ is applied at initialization and the final iterates are rescaled. In contrast to the standard one-sequence Newton-Schulz iteration, which can be obtained by eliminating one variable from the coupled Newton iteration under a commuting initialization, the coupled formulation simultaneously evaluates $\mathbf{Y}_k$ and $\mathbf{Z}_k$, making it a more suitable tool for computing matrix roots and inverse roots. Algorithm 1 depicts our method.

4. Experiments

4.1. Experimental Settings

Setup. We conduct our experiments on nanoGPT¹ (Karpathy, 2022), a lightweight GPT-2 training codebase. We follow the standard GPT-2 configuration: GELU activations and a byte-pair encoding tokenizer with vocabulary size 50,257. Our GPT-2 model has 124M parameters. All runs use sequence length 1024, global batch size 480, and are trained on OpenWebText for 200K optimization steps with warmup for 2,000 steps. We do not use QK-Norm or QK-Clip for our primary experiments and conduct all Newton–Schulz iterations with iteration count $K = 5$. Across runs, we vary only the optimizer and its learning rate.

For Muon, we use the reference implementation² from Jordan et al. (2024), which applies the matrix update $\mathbf{W}_{t+1} = \mathbf{W}_t - \eta_t \sqrt{\frac{\text{fan-out}}{\text{fan-in}}} \mathbf{U}\mathbf{V}^\top$, where $\mathbf{U}\mathbf{V}^\top$ is the polar factor computed via Algorithm 2.

Controlled comparison. To ensure fair and interpretable comparisons, we isolate the optimizer’s effect on *matrix-shaped* parameters and, crucially, *decouple matrix and vector learning rates for all optimizers*.

Muon is typically evaluated with separate learning rates for matrix and vector parameters, whereas standard Adam baselines often share a single global learning rate. To eliminate this mismatch, we follow Muon and apply Adam to all vector-valued parameters for every method using a fixed learning rate $\text{lr}_{\text{vec}} = 3 \times 10^{-4}$. For matrix parameters, each optimizer is tuned with its own learning rate lr_{mat} .

We also disable weight decay in the main experiments to isolate the effect of the matrix update rule. Since weight decay enters the update as $\mathbf{W}_{t+1} = \mathbf{W}_t - \eta_t \Delta \mathbf{W}_t - \eta_t \lambda \mathbf{W}_t$, jointly tuning (η_t, λ) would introduce an additional degree of freedom and complicate controlled comparisons. We therefore omit weight decay from the main experiments, and later provide an ablation after fixing the learning rate. We also avoid auxiliary stabilization techniques such as QK-Norm and QK-Clip. All remaining hyperparameters are fixed across methods; in particular, $\beta_1 = 0.9$ and $\beta_2 = 0.95$.

4.2. Learning Rate Tuning Protocol

We tune the matrix-parameter learning rate lr_{mat} for each optimizer using a two-stage procedure: a coarse logarithmic sweep to identify a stable scale, followed by a local refinement within that scale.

Coarse search. We first sweep lr_{mat} on a logarithmic grid spanning several orders of magnitude to locate the re-

gion where the optimizer transitions from divergence to effective learning. For instance, for AdamW we evaluate $\{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$. This stage intends to identify the correct order of magnitude rather than the exact optimum. Since most candidates in this sweep diverge, we do not plot the resulting curves.

Fine-grained search. After identifying a promising scale, we refine the search by evaluating a denser set of learning rates above and below the best coarse candidate. We stop refining once the selected learning rate performs better than its immediate neighbors in this local grid, indicating a stable local optimum within the explored range. For each optimizer, this refinement evaluates roughly ten candidates; for readability, we plot six representative runs in the figures.

4.3. Effect of Spectral Exponent on Momentum Inputs

Figure 2 reports representative tuning results when the input $\mathbf{O}_t^{\text{mom}}$ is used. We make three observations:

- *Muon stabilizes momentum updates.* Muon (mSGDZ) is considerably more stable than mSGD across a wide learning-rate range. The best learning rate we find for Muon, 7×10^{-3} , closely matches the value reported in prior benchmarking (Wen et al., 2025).
- *Partial compression improves stability but not to Muon’s level.* The intermediate variants mSGDS and mSGDQ are more stable than mSGD, but remain less robust than Muon under aggressive learning rates.
- *After stability is achieved, moderate compression can outperform flattening.* Among the four momentum-input variants, mSGDQ attains the strongest performance once it trains stably, exceeding Muon and obtaining strong results at learning rates of 4×10^{-2} and 5×10^{-2} .

4.4. Effect of Spectral Exponent on Normalized Inputs

Similarly, Figure 3 shows our selected results given rms-normalized momentum $\mathbf{O}_t^{\text{rms}}$ as input. We observe that:

- *Spectral transforms yield smaller gains when the input is already normalized.* Applying spectral compression to $\mathbf{O}_t^{\text{rms}}$ produces limited improvements: AdamS can modestly broaden the stable learning-rate range, whereas stronger compression (AdamQ) offers little benefit and full flattening (AdamZ) degrades performance.
- *All four variants behave similarly at their best settings.* The peak performance differences among Adam, AdamS, and AdamQ are modest, suggesting that elementwise normalization already controls much of the harmful anisotropy that spectral compression targets.

¹<https://github.com/karpathy/nanoGPT>

²<https://github.com/KellerJordan/Muon>

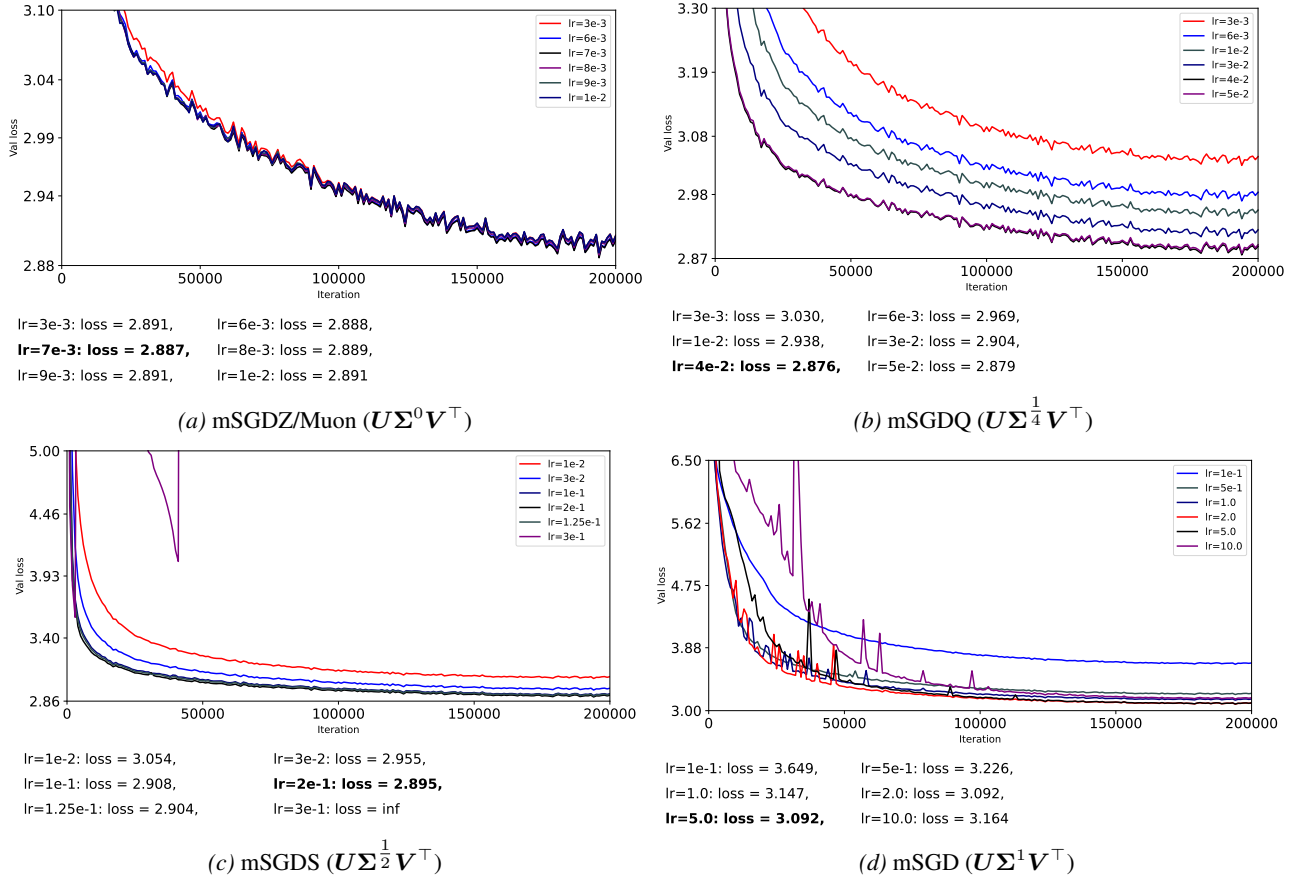


Figure 2. Overall comparison across four optimizers (mSGDZ/Muon, mSGDQ, mSGDS and mSGD) based on the **first-moment momentum** M_t . Each subfigure corresponds to a different optimizer.

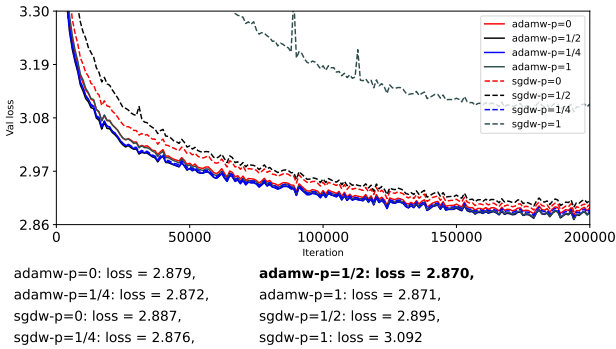


Figure 4. Comparison of four spectral exponents $p \in \{0, \frac{1}{4}, \frac{1}{2}, 1\}$ applied to either the first-moment momentum M_t or the RMS-normalized update $M_t \propto \sqrt{V_t}$, each shown at its tuned learning rate. Dashed curves correspond to momentum-input variants, while solid curves correspond to RMS-normalized-input variants.

4.5. Momentum vs. RMS-Normalized Inputs

In Figure 4, we compare the best-tuned run from each of the eight configurations. Two trends are clear. First, switching the input from the momentum O_t^{mom} to the RMS-normalized

update O_t^{rms} consistently improves both stability and final performance across spectral exponents. In particular, AdamZ (Muon-style orthogonalization applied to O_t^{rms}) is stronger than mSGDZ (the original Muon applied to O_t^{mom}). Second, within the RMS-normalized family, performance differences across p are relatively small, with AdamS emerging as a strong and stable choice.

4.6. Additional Ablations and Comparisons

Comparison under weight decay and QK-Norm. Our main experiments disable weight decay and auxiliary stabilization techniques to isolate the effect of the update rule. To determine whether our conclusions persist under more practical settings, we additionally evaluate the RMS-normalized spectral variants with weight decay and QK-Norm enabled. The full training curves are provided in Appendix E.

Since weight decay enters the parameter update through the coupled product $\eta\lambda$, we start from the best learning rate selected for each Adam variant in Figure 3, and choose the corresponding λ such that $\eta\lambda$ matches the Adam baseline value $4 \times 10^{-3} \cdot 0.1$.

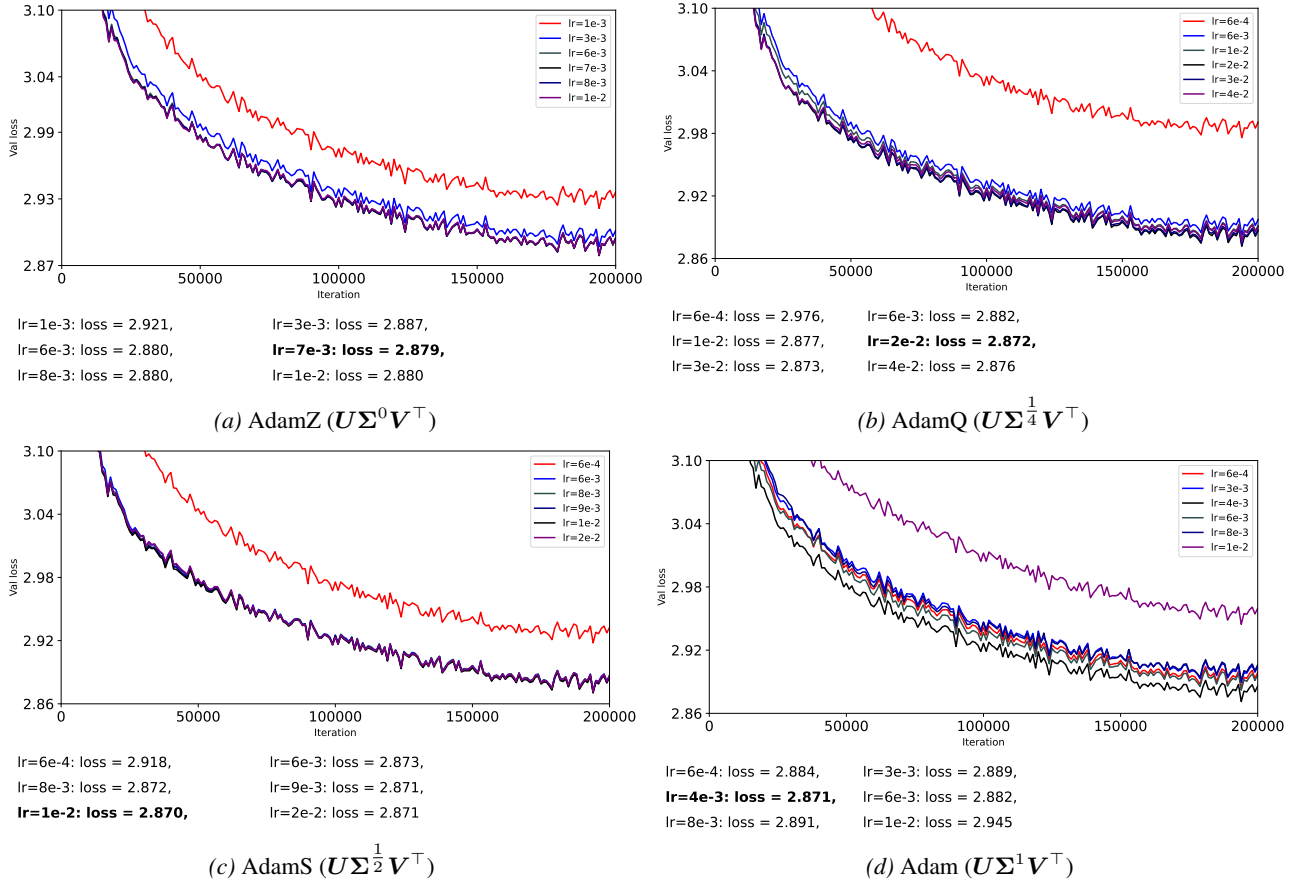


Figure 3. Overall comparison across four optimizers (AdamZ, AdamS, AdamQ and Adam) based on the **second-moment-normalized update** $M_t \oslash \sqrt{V_t}$. Each subfigure corresponds to a different optimizer.

With both weight decay and QK-Norm, AdamZ, AdamS, AdamQ, and Adam reach final validation losses of 2.806, 2.802, 2.800, and 2.815, respectively. Weight decay improves all RMS-normalized variants, and QK-Norm provides a further gain. Importantly, the relative trend remains consistent with our main finding: spectral compression offers useful stabilization and performance benefits, but full spectral flattening is not uniformly superior, especially once RMS normalization and other stabilizers are incorporated.

Effect of Newton–Schulz iteration count. We also study the sensitivity of the spectral transform to the number of Newton–Schulz iterations in [Appendix F](#). Even with synthetic inputs that span a large range of singular values, the coupled NS iteration algorithm with $K = 5$ yields results with high cosine similarity to those from direct SVD.

Moreover, on our experimental setup, increasing the iteration count from $K = 5$ to $K = 15$ produces nearly identical training curves and final validation losses. This suggests that $K = 5$ is sufficient for the optimizer comparisons reported in the main experiments.

4.7. Runtime Overhead of Newton–Schulz Iterations

Newton–Schulz iterations introduce only moderate training overhead in our setting. For $W \in \mathbb{R}^{m \times n}$ with $n \leq m$, a K -step NS transform costs $O(Kn^3 + mn^2)$, where Kn^3 comes from repeated operations on the smaller $n \times n$ Gram matrix, and mn^2 comes from forming the Gram matrix and multiplying the spectral factor back to the original update.

Importantly, this cost is independent of the batch size per step, so its relative contribution to the overall training cost decreases as the batch size increases. The main additional memory overhead comes from transient $n \times n$ matrices. With $K = 5$ on an 8-GPU A800 node, the wall-clock overhead across Adam spectral variants is shown in [Table 1](#).

Table 1. Runtime overhead on an 8-GPU A800 node.

	Adam	AdamS	AdamQ	AdamZ
Runtime	33h 05m	34h 18m	34h 21m	34h 09m
Overhead	–	3.7%	3.8%	3.2%

5. Discussion: Our Understanding of Muon

The primary goal of this work is to evaluate when Muon-style orthogonalization (and, more broadly, spectral compression) is a reliable optimization strategy. Accordingly, we interpret Muon’s empirical behavior in Section 4 through three honest conclusions.

I. Muon exhibits no significant performance edge over AdamW in our experimental regime. In our controlled setting, Muon-style orthogonalization does not consistently outperform Adam, whether applied to first-moment momentum or to RMS-normalized updates. This contrasts with prior reports of Muon converging faster than Adam (Jordan et al., 2024). The discrepancy is likely due to differences in methodology, which include learning-rate decoupling, omission of auxiliary stabilizers, disabled weight decay, and a relatively small 124M-parameter model scale.

II. Muon is stable but aggressive. On first-moment momentum inputs, Muon provides a clear stabilization effect where orthogonalization significantly improves robustness over mSGD across most learning rates (Figure 2). This behavior aligns with Muon’s defining operation: given $G_t = U\Sigma V^\top$, Muon replaces the spectrum by Σ^0 , producing the polar factor $UV^\top$. By discarding singular-value magnitudes, Muon prevents directions with extremely large singular values from dominating the update, mitigating ill-conditioning through *direction-wise* spectral scale control.

However, this stabilization mechanism is also indiscriminate. Muon applies the same singular-value scaling to every direction, independent of curvature or signal strength. While momentum scale is not necessarily an optimal step-size signal, full spectral flattening removes all magnitude distinctions among singular directions. Thus, when some directions are legitimately more informative than others, flattening them to unity can reduce their relative influence. In this sense, Muon stabilizes by construction, but it does not selectively preserve meaningful anisotropy when such anisotropy may be beneficial.

III. Muon may magnify noisy gradient directions. Muon’s biggest limitation also follows from its complete spectral flattening: by mapping all nonzero singular values to 1, it removes magnitude-based filtering. Directions corresponding to small singular values are no longer suppressed relative to dominant directions and can receive comparable update weight. This is especially problematic in the later stages of training where update matrices typically have low effective rank, and small-singular-value components may be primarily dominated by noise.

This effect is consistent with our observation that aggressive Muon-style orthogonalization on RMS-normalized inputs

does not consistently outperform Adam and is often worse than partial compression, which can offer a more balanced trade-off between stabilization and preserving original spectral structure.

Overall. Together, our results suggest that spectral compression can provide genuine stabilization benefits, especially when the input update is *not* already normalized. However, full orthogonalization ($p = 0$) is not always desirable: its indiscriminate flattening can suppress informative dominant modes while amplifying low-signal modes. Hence, our experiments do not support orthogonalization as a universally superior replacement for modern RMS-normalized second-moment optimizers.

6. Conclusion and Limitations

Conclusion. This paper studies Muon and related matrix-based optimizers from a unified spectral perspective. By viewing Muon as the $p = 0$ endpoint of the family $\Psi_p(\mathcal{O}) = U\Sigma^p V^\top$, we separate the effect of spectral compression from other optimizer design choices. Our controlled experiments show that spectral compression provides a clear stabilization benefit for first-moment updates, but full Muon-style flattening is not uniformly superior once RMS-normalized second-moment updates are used. These results suggest that Muon is best understood as an effective spectral normalization mechanism rather than a universally superior replacement for Adam-style optimizers.

Limitations and future work. Our study is intentionally conducted in a controlled nanoGPT/OpenWebText regime to isolate the effect of spectral transformations from confounding factors such as weight decay, auxiliary normalization, and optimizer-specific tuning. While this design makes the comparison more interpretable, it also limits the scope of our empirical conclusions. In particular, we do not claim that the relative ranking of optimizers will necessarily hold across larger model scales, different architectures, longer training runs, or fully tuned production training recipes.

Although our ablations include more practical ingredients such as weight decay and QK-Norm, which are omitted from the main experiments, these studies are still limited in scope. A more complete investigation of how spectral optimizers interact with weight decay, QK-Norm, learning-rate decay, and other stabilization techniques is an important direction for future work.

Finally, fractional spectral variants rely on coupled Newton–Schulz iterations, which introduce moderate runtime and memory overhead. Improving the efficiency of these matrix-function approximations is therefore another important direction for making spectral compression more practical at larger scales.

Acknowledgment

This work is partly supported by the National Natural Science Foundation of China under Grant No. 62576048.

Impact Statement

“This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.”

References

- Ba, J. L., Kiros, J. R., and Hinton, G. E. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Bernstein, J. Deriving muon, 2025a. URL <https://jeremybernste.in/writing/deriving-muon>.
- Bernstein, J. Modular manifolds. *Thinking Machines Lab: Connectionism*, 2025b. doi: 10.64434/tml.20250926. <https://thinkingmachines.ai/blog/modular-manifolds/>.
- Bernstein, J., Wang, Y.-X., Azizzadenesheli, K., and Anandkumar, A. signsgd: Compressed optimisation for non-convex problems. In *International Conference on Machine Learning*, pp. 560–569. PMLR, 2018.
- Bottou, L., Curtis, F. E., and Nocedal, J. Optimization methods for large-scale machine learning. *SIAM review*, 60(2):223–311, 2018.
- Buchanan, S. A faster manifold muon with ADMM. <https://sdbuchanan.com/blog/manifold-muon/>, 2025.
- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Liu, Y., Pham, H., Dong, X., Luong, T., Hsieh, C.-J., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms, 2023. URL <https://arxiv.org/abs/2302.06675>.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv e-prints*, pp. arXiv–2407, 2024.
- Duchi, J., Hazan, E., and Singer, Y. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7), 2011.
- Frans, K., Abbeel, P., and Levine, S. What really matters in matrix-whitening optimizers? *arXiv preprint arXiv:2510.25000*, 2025.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Gupta, V., Koren, T., and Singer, Y. Shampoo: Pre-conditioned stochastic tensor optimization. In *International Conference on Machine Learning*, pp. 1842–1850. PMLR, 2018.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Higham, N. J. Computing the polar decomposition—with applications. *SIAM Journal on Scientific and Statistical Computing*, 7(4):1160–1174, 1986.
- Higham, N. J. Computing real square roots of a real matrix. *Linear Algebra and its applications*, 88:405–430, 1987.
- Higham, N. J. *Functions of matrices: theory and computation*. SIAM, 2008.
- Hinton, G., Srivastava, N., and Swersky, K. Neural networks for machine learning: Lecture 6a, overview of mini-batch gradient descent. Coursera lecture slides, 2012. URL <https://www.cs.toronto.edu/~hinton/coursera/lecture6/lec6.pdf>.
- Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- Horn, R. A. and Johnson, C. R. *Matrix analysis*. Cambridge university press, 2012.
- Johnson, R. and Zhang, T. Accelerating stochastic gradient descent using predictive variance reduction. *Advances in neural information processing systems*, 26, 2013.
- Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- Karpathy, A. NanoGPT. <https://github.com/karpathy/nanoGPT>, 2022.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Lau, T. T.-K., Long, Q., and Su, W. Polargrad: A class of matrix-gradient optimizers from a unifying preconditioning perspective. *arXiv preprint arXiv:2505.21799*, 2025.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 2002.

- Liang, K., Chen, L., Liu, B., and Liu, Q. Cautious optimizers: Improving training with one line of code, 2025. URL <https://arxiv.org/abs/2411.16085>.
- Liu, H., Li, Z., Hall, D., Liang, P., and Ma, T. Sophia: A scalable stochastic second-order optimizer for language model pre-training, 2024. URL <https://arxiv.org/abs/2305.14342>.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Martens, J. and Grosse, R. Optimizing neural networks with kronecker-factored approximate curvature. In *International conference on machine learning*, pp. 2408–2417. PMLR, 2015.
- Nesterov, Y. A method for unconstrained convex minimization problem with the rate of convergence $O(1/k^2)$. In *Doklady an ussr*, volume 269, pp. 543–547, 1983.
- Newhouse, L. Understanding muon, 2025. URL <https://lakernewhouse.com/muon>.
- Qi, X., Wang, J., Chen, Y., Shi, Y., and Zhang, L. Lipsformer: Introducing lipschitz continuity to vision transformers. *arXiv preprint arXiv:2304.09856*, 2023a.
- Qi, X., Wang, J., and Zhang, L. Understanding optimization of deep learning via jacobian matrix and lipschitz constant. *arXiv preprint arXiv:2306.09338*, 2023b.
- Robbins, H. and Monro, S. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3): 400–407, 1951. ISSN 00034851. URL <http://www.jstor.org/stable/2236626>.
- Schulz, G. Iterative berechnung der reziproken matrix. *ZAMM-Journal of Applied Mathematics and Mechanics/Zeitschrift für Angewandte Mathematik und Mechanik*, 13(1):57–59, 1933.
- Shazeer, N. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- Shazeer, N. and Stern, M. Adafactor: Adaptive learning rates with sublinear memory cost. In *International Conference on Machine Learning*, pp. 4596–4604. PMLR, 2018.
- Si, C., Zhang, D., and Shen, W. Adamuon: Adaptive muon optimizer, 2025. URL <https://arxiv.org/abs/2507.11005>.
- Su, W. Isotropic curvature model for understanding deep learning optimization: Is gradient orthogonalization optimal? *arXiv preprint arXiv:2511.00674*, 2025.
- Team, K., Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023a.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023b.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Wen, K., Hall, D., Ma, T., and Liang, P. Fantastic pretraining optimizers and where to find them. *arXiv preprint arXiv:2509.02046*, 2025.
- Xie, X., Zhou, P., Li, H., Lin, Z., and Yan, S. Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models, 2024. URL <https://arxiv.org/abs/2208.06677>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- You, Y., Li, J., Reddi, S., Hseu, J., Kumar, S., Bhojanapalli, S., Song, X., Demmel, J., Keutzer, K., and Hsieh, C.-J. Large batch optimization for deep learning: Training bert in 76 minutes. *arXiv preprint arXiv:1904.00962*, 2019.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2025. URL <https://arxiv.org/abs/2411.10438>.
- Zhang, Y., Chen, C., Li, Z., Ding, T., Wu, C., Kingma, D. P., Ye, Y., Luo, Z.-Q., and Sun, R. Adam-mini: Use fewer learning rates to gain more, 2025. URL <https://arxiv.org/abs/2406.16793>.

Zur, J. Matrix functions. Technical report,
Technische Universität Berlin, 2025. URL
[https://www.tu.berlin/fileadmin/www/
40000110/Dauer-Wimis/Jan_Zur/matrix_
functions_zur.pdf](https://www.tu.berlin/fileadmin/www/40000110/Dauer-Wimis/Jan_Zur/matrix_functions_zur.pdf). Lecture note.

Almost all derivations are about functions of matrices, all these derivatinsos can be found in the following materials (Higham, 1986; 1987; 2008; Zur, 2025; Horn & Johnson, 2012).

A. Derivation of $UV^\top$ based on Newton-Schulz method

Let the singular value decomposition of a matrix $M \in \mathbb{R}^{m \times n}$ be

$$M = USV^\top,$$

where $U \in \mathbb{R}^{m \times r}$, $V \in \mathbb{R}^{n \times r}$, $S = \text{Diag}(\sigma_1, \dots, \sigma_r)$, $\sigma_i > 0$, and U, V have orthonormal columns. Let us assume $m \leq n$

Our goal is to compute the orthogonal (polar) factor $UV^\top$ *without explicitly performing an SVD*, using the Newton–Schulz iteration.

Step 1: Express $UV^\top$ as a matrix function of M . We start from quantities that can be formed directly from M . Consider $M^\top M \in \mathbb{R}^{n \times n}$. Substituting the SVD of M gives

$$M^\top M = (USV^\top)^\top (USV^\top) = VSU^\top USV^\top = VS^2V^\top,$$

where we used $U^\top U = I_r$.

Hence, the inverse square root of $M^\top M$ (on its rank- r subspace) is

$$(M^\top M)^{-1/2} = VS^{-1}V^\top.$$

Left-multiplying by M yields

$$M(M^\top M)^{-1/2} = (USV^\top)(VS^{-1}V^\top) = U(SS^{-1})V^\top = UV^\top.$$

This identity shows that $UV^\top$ can be obtained by removing the singular-value scaling of M via $(M^\top M)^{-1/2}$.

Step 2: Reduction to an inverse square root problem. Define

$$A := M^\top M \succeq 0.$$

The problem is now reduced to computing $A^{-1/2}$ using only matrix multiplication, without SVD.

Step 3: Newton–Schulz iteration for the inverse square root. Let us consider a scalar $a > 0$, consider the problem of computing $z = a^{-1/2}$. This is equivalent to solving $f(z) = \frac{1}{z^2} - a = 0$. Applying Newton’s method gives

$$z^+ = z - \frac{f(z)}{f'(z)} = z - \frac{z^{-2} - a}{-2z^{-3}} = \frac{1}{2}z(3 - az^2).$$

This is the scalar Newton–Schulz update.

Replacing the scalar a by a matrix $A \succeq 0$, the scalar z by a matrix Z , and scalar multiplication by matrix multiplication yields the matrix Newton–Schulz iteration

$$Z_{k+1} = \frac{1}{2}Z_k(3I - AZ_k^2).$$

This iteration is equal to Equation 4 in the main body. It involves only matrix multiplication, addition, and scalar scaling. After sufficient iterations,

$$Z_k \approx A^{-1/2}.$$

Step 4: Recovering $UV^\top$. Using the identity $UV^\top = MA^{-1/2}$, $A = M^\top M$, and the approximation $Z_k \approx A^{-1/2}$, we obtain

$$UV^\top \approx MZ_k.$$

B. Newton-Schulz method used by Jordan et al. (2024)

Algorithm 2 Newton-Schulz to solve $UV^\top$ where $M = U\Sigma V^\top$

Require: M, K

Ensure: $UV^\top$

```

1: Coefficients:  $(a, b, c) \leftarrow (3.4445, -4.7750, 2.0315)$ 
2:  $Z_0 \leftarrow M$ 
3:  $\alpha \leftarrow \|Z_0\|_F$ 
4:  $Z_0 \leftarrow Z_0/\alpha$ ,  $k \leftarrow 0$ 
5: while  $k < K$  do
6:    $Z_{k+1} \leftarrow aZ_k + bZ_kZ_k^\top Z_k + c(Z_kZ_k^\top)^2 Z_k$ 
7:    $k \leftarrow k + 1$ 
8: end while
9: return  $Z_K$ 
    
```

C. Derive $X^{\frac{1}{2}}$ based on Coupled Newton-Schulz method

We aim to compute the matrix square root $X^{1/2}$. Let the unknown matrix Y satisfy

$$f(Y) := Y^2 - X = 0.$$

This defines a matrix equation $f : \mathbb{C}^{n \times n} \rightarrow \mathbb{C}^{n \times n}$.

Fréchet Derivative. For an arbitrary perturbation H , we have

$$f(Y + H) = (Y + H)^2 - X = Y^2 - X + (YH + HY) + H^2.$$

Therefore, the Fréchet derivative of f at Y is

$$Df(Y)[H] = YH + HY.$$

Definition of the Newton Step. The Newton increment ΔY at Y is defined as the solution of

$$Df(Y)[\Delta Y] = -f(Y),$$

that is, the Sylvester equation

$$Y\Delta Y + \Delta Y Y = -(Y^2 - X). \tag{9}$$

The update is then given by

$$Y^+ = Y + \Delta Y.$$

Up to this point, the derivation corresponds to the *exact matrix Newton method*, without any approximation.

A Coupled Newton System. The main difficulty of (9) is that it requires solving a Sylvester equation. A classical technique is to couple the unknown square root with its inverse, thereby converting the Sylvester solve into matrix multiplications.

Let Z approximate Y^{-1} (and ultimately $X^{-1/2}$). Consider the coupled system

$$\begin{cases} F_1(Y, Z) := Y^2 - X = 0, \\ F_2(Y, Z) := ZY - I = 0. \end{cases} \quad (10)$$

The exact solution is

$$(Y_*, Z_*) = (X^{1/2}, X^{-1/2}).$$

Fréchet Derivative of the Coupled System For perturbations $(\Delta Y, \Delta Z)$:

- For F_1 ,

$$DF_1(Y, Z)[\Delta Y, \Delta Z] = Y\Delta Y + \Delta Y Y.$$

- For F_2 ,

$$(Z + \Delta Z)(Y + \Delta Y) - I = (ZY - I) + (Z\Delta Y + \Delta Z Y) + (\Delta Z)(\Delta Y),$$

hence

$$DF_2(Y, Z)[\Delta Y, \Delta Z] = Z\Delta Y + \Delta Z Y.$$

Newton Linear System (Exact) The Newton step $(\Delta Y, \Delta Z)$ is defined by

$$DF(Y, Z)[\Delta Y, \Delta Z] = -F(Y, Z),$$

that is,

$$\begin{aligned} Y\Delta Y + \Delta Y Y &= -(Y^2 - X), \\ Z\Delta Y + \Delta Z Y &= -(ZY - I). \end{aligned} \quad (11)$$

This system is still an *exact Newton system*.

A Closed-Form Newton Step via a Symmetric Multiplicative Ansatz We adopt the symmetric multiplicative ansatz

$$Y^+ = YQ, \quad Z^+ = QZ, \quad (12)$$

where $Q = Q(Y, Z)$ is to be determined.

Define

$$R := ZY.$$

Then

$$R^+ := Z^+ Y^+ = (QZ)(YQ) = Q(ZY)Q = QRQ.$$

Hence, choosing Q reduces to the problem: given R , construct Q such that $QRQ \approx I$.

Deriving $Q = \frac{1}{2}(3I - ZY)$ from Newton's Principle. To approximate $R^{-1/2}$, ideally,

$$Q = R^{-1/2} \Rightarrow QRQ = I.$$

Consider the matrix function

$$\phi(R) := R^{-1/2}.$$

When R is close to the identity, write

$$R = I + E, \quad \|E\| \ll 1.$$

According to the Taylor expansion, we have

$$(I + E)^{-1/2} = I - \frac{1}{2}E + O(\|E\|^2).$$

Substituting $E = R - I$, we obtain

$$R^{-1/2} \approx I - \frac{1}{2}(R - I) = \frac{1}{2}(3I - R).$$

Thus define

$$Q := \frac{1}{2}(3I - R) = \frac{1}{2}(3I - ZY). \quad (13)$$

Deriving $Q = \frac{1}{2}(3I - ZY)$ from Newton's Principle Coupled Newton–Schulz Iteration.

Substituting (13) into (12), we obtain

$$\begin{aligned} Y^+ &= \frac{1}{2} Y(3I - ZY), \\ Z^+ &= \frac{1}{2} (3I - ZY) Z. \end{aligned}$$

Write it as an iteration equation, we have

$$\begin{aligned} Y_{k+1} &= \frac{1}{2} Y_k(3I - Z_k Y_k), & Y_0 &= X, \\ Z_{k+1} &= \frac{1}{2} (3I - Z_k Y_k) Z_k, & Z_0 &= I. \end{aligned} \quad (14)$$

This is the *coupled Newton–Schulz iteration*, which simultaneously drives

$$Y_k \rightarrow X^{1/2}, \quad Z_k \rightarrow X^{-1/2}.$$

D. Derive $X^{\frac{1}{4}}$ based on Coupled Newton-Schulz method

Algorithm 3 Two-times Coupled Newton-Schulz to solve $X^{\frac{1}{4}}$ and $X^{\frac{-1}{4}}$

Require: X, K

Ensure: $X^{\frac{1}{4}}, X^{\frac{-1}{4}}$

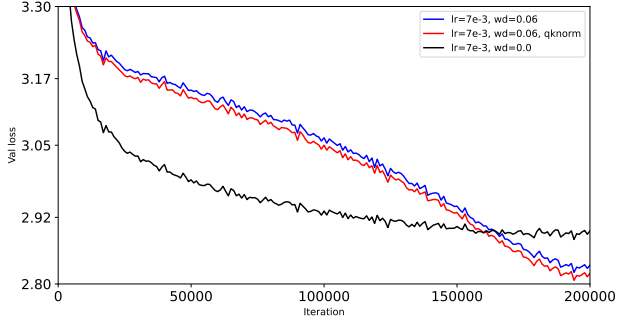
```

1:  $Y_0 \leftarrow X, Z_0 \leftarrow I$ 
2:  $\alpha \leftarrow \|X\|_F$ 
3:  $Y_0 \leftarrow \frac{Y_0}{\alpha}, k \leftarrow 0$ 
4: while  $k < K$  do
5:    $T_k \leftarrow 3I - Z_k Y_k$ 
6:    $Y_{k+1} \leftarrow \frac{1}{2} Y_k T_k$ 
7:    $Z_{k+1} \leftarrow \frac{1}{2} T_k Z_k$ 
8:    $k \leftarrow k + 1$ 
9: end while
10:  $Y_0 \leftarrow \sqrt{\alpha} Y_K, Z_0 \leftarrow I$ 
11:  $\beta \leftarrow \|Y_0\|_F$ 
12:  $Y_0 \leftarrow \frac{Y_0}{\beta}, k \leftarrow 0$ 
13: while  $k < K$  do
14:    $T_k \leftarrow 3I - Z_k Y_k$ 
15:    $Y_{k+1} \leftarrow \frac{1}{2} Y_k T_k$ 
16:    $Z_{k+1} \leftarrow \frac{1}{2} T_k Z_k$ 
17:    $k \leftarrow k + 1$ 
18: end while
19: return  $\sqrt{\beta} Y_K, \frac{1}{\sqrt{\beta}} Z_K$ 

```

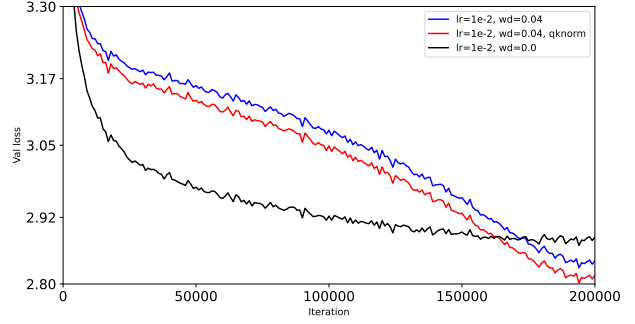
E. Additional Comparisons under Weight Decay and QK-Norm

We provide the full training curves for the additional comparisons with weight decay and QK-Norm in Figure 5.



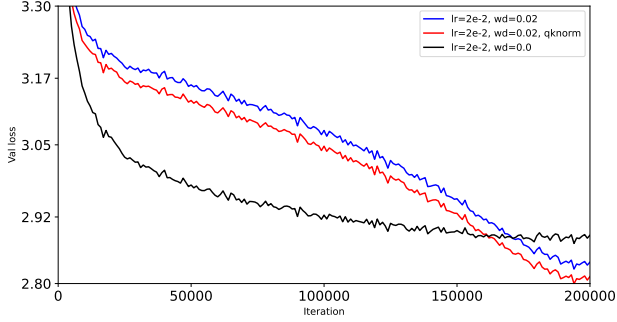
$lr=7e-3, wd=0.06$: loss = 2.821, $lr=7e-3, wd=0.06, qknorm$: loss = 2.806,
 $lr=7e-3, wd=0.0$: loss = 2.880

(a) AdamZ ($p = 0$).



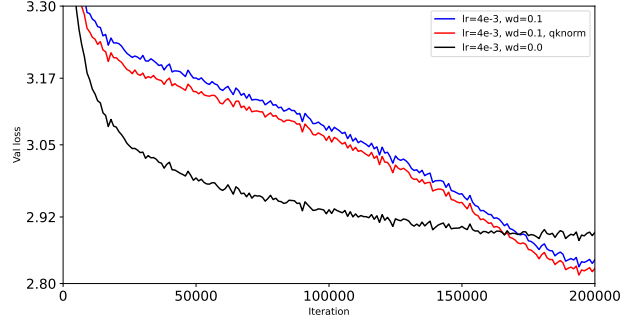
$lr=1e-2, wd=0.04$: loss = 2.829, $lr=1e-2, wd=0.04, qknorm$: loss = 2.802,
 $lr=1e-2, wd=0.0$: loss = 2.869

(b) AdamS ($p = \frac{1}{2}$).



$lr=2e-2, wd=0.02$: loss = 2.826, $lr=2e-2, wd=0.02, qknorm$: loss = 2.800,
 $lr=2e-2, wd=0.0$: loss = 2.872

(c) AdamQ ($p = \frac{1}{4}$).



$lr=4e-3, wd=0.1$: loss = 2.830, $lr=4e-3, wd=0.1, qknorm$: loss = 2.815,
 $lr=4e-3, wd=0.0$: loss = 2.878

(d) Adam ($p = 1$).

Figure 5. Additional comparisons under weight decay and QK-Norm across RMS-normalized spectral variants. Each panel compares training without weight decay, with weight decay, and with both weight decay and QK-Norm.

F. Ablation of Newton–Schulz Iteration Count

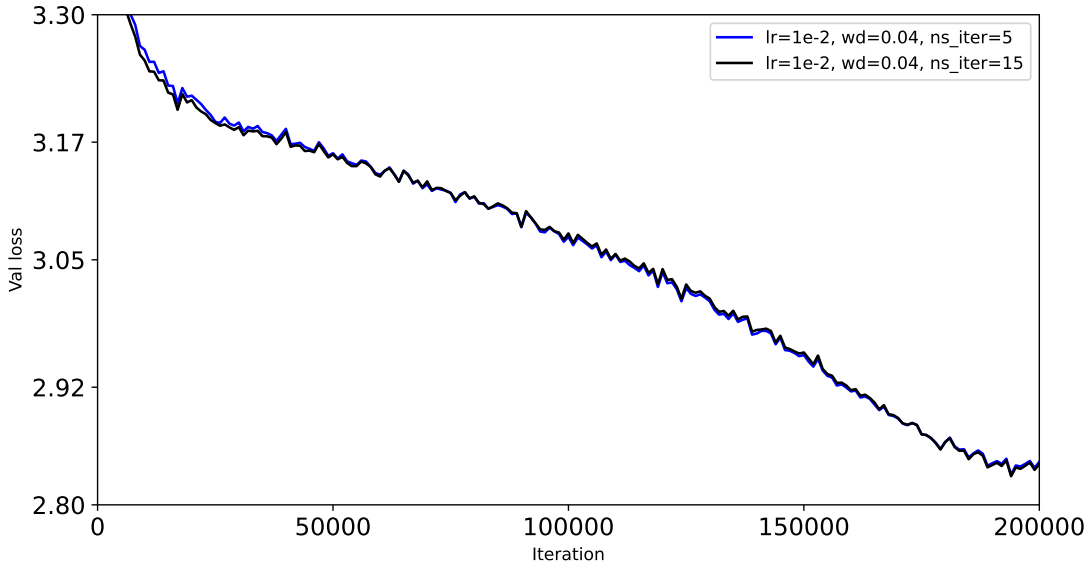
To evaluate the approximation quality of the coupled Newton–Schulz (NS) procedure, we compare the NS approximation against the exact SVD-based spectral transform. Specifically, we generate Gaussian matrices of sizes 512×512 and 1024×1024 with condition numbers ranging from 1 to 100. For each matrix $\mathbf{O}$, we compute the exact update $\Psi_p(\mathbf{O})$ using SVD and compare it with the K -step NS approximation using cosine similarity between the vectorized update matrices. Representative results are shown in Table 2.

Table 2. Cosine similarity between NS approximation and the exact SVD-based spectral transform for synthetic inputs.

K	cos-sim ($p = \frac{1}{2}$)	cos-sim ($p = \frac{1}{4}$)
3	0.990	0.975
5	0.997	0.993
7	0.999	0.999
10	1.000	1.000

The approximation improves rapidly as K increases. By $K = 5$, both $p = \frac{1}{2}$ and $p = \frac{1}{4}$ already achieve very high cosine similarity with the exact SVD solution. Therefore, we use $K = 5$ in our main experiments.

We also verify whether increasing the number of NS iterations affects actual training behavior. In Figure 6, we compare AdamS ($p = \frac{1}{2}$) with $K = 5$ and $K = 15$ under the same setting. The two training curves are nearly identical, and the final validation losses are also very close.



lr=1e-2, wd=0.04, ns_iter=5: loss = 2.831,

lr=1e-2, wd=0.04, ns_iter=15: loss = 2.829

Figure 6. Effect of Newton–Schulz iteration count on training for AdamS ($p = \frac{1}{2}$) with $K = 5$ and $K = 15$ NS iterations.

G. Detailed results for different settings

Table 3. Hyperparameter sweeps for Muon.

Origin	$\Psi_p(O^{\text{mom}})$	Learning Rate	Weight Decay	Val Loss
First order	$U\Sigma^1 V^\top$	1e-1	0	3.649
First order	$U\Sigma^1 V^\top$	5e-1	0	3.226
First order	$U\Sigma^1 V^\top$	1.0	0	3.147
First order	$U\Sigma^1 V^\top$	2.0	0	3.092
First order	$U\Sigma^1 V^\top$	5.0	0	3.092
First order	$U\Sigma^1 V^\top$	10.0	0	3.164
First order	$U\Sigma^{\frac{1}{2}} V^\top$	1e-2	0	3.054
First order	$U\Sigma^{\frac{1}{2}} V^\top$	3e-2	0	2.955
First order	$U\Sigma^{\frac{1}{2}} V^\top$	1e-1	0	2.908
First order	$U\Sigma^{\frac{1}{2}} V^\top$	2e-1	0	2.895
First order	$U\Sigma^{\frac{1}{2}} V^\top$	1.25e-1	0	2.904
First order	$U\Sigma^{\frac{1}{2}} V^\top$	3e-1	0	inf
First order	$U\Sigma^{\frac{1}{4}} V^\top$	3e-3	0	3.030
First order	$U\Sigma^{\frac{1}{4}} V^\top$	6e-3	0	2.969
First order	$U\Sigma^{\frac{1}{4}} V^\top$	1e-2	0	2.938
First order	$U\Sigma^{\frac{1}{4}} V^\top$	3e-2	0	2.904
First order	$U\Sigma^{\frac{1}{4}} V^\top$	4e-2	0	2.876
First order	$U\Sigma^{\frac{1}{4}} V^\top$	5e-2	0	2.879
First order	$U\Sigma^0 V^\top$	3e-3	0	2.891
First order	$U\Sigma^0 V^\top$	6e-3	0	2.888
First order	$U\Sigma^0 V^\top$	7e-3	0	2.887
First order	$U\Sigma^0 V^\top$	8e-3	0	2.889
First order	$U\Sigma^0 V^\top$	9e-3	0	2.891
First order	$U\Sigma^0 V^\top$	1e-2	0	2.891

Table 4. Hyperparameter sweeps for Muon.

Origin	$\Psi_p(O^{\text{rms}})$	Learning Rate	Weight Decay	Val Loss
Second order	$U\Sigma^1 V^\top$	6e-4	0	2.884
Second order	$U\Sigma^1 V^\top$	3e-3	0	2.889
Second order	$U\Sigma^1 V^\top$	4e-3	0	2.871
Second order	$U\Sigma^1 V^\top$	6e-3	0	2.882
Second order	$U\Sigma^1 V^\top$	8e-3	0	2.891
Second order	$U\Sigma^1 V^\top$	1e-2	0	2.945
Second order	$U\Sigma^{\frac{1}{2}} V^\top$	6e-4	0	2.918
Second order	$U\Sigma^{\frac{1}{2}} V^\top$	6e-3	0	2.873
Second order	$U\Sigma^{\frac{1}{2}} V^\top$	8e-3	0	2.872
Second order	$U\Sigma^{\frac{1}{2}} V^\top$	9e-3	0	2.871
Second order	$U\Sigma^{\frac{1}{2}} V^\top$	1e-2	0	2.870
Second order	$U\Sigma^{\frac{1}{2}} V^\top$	2e-2	0	2.871
Second order	$U\Sigma^{\frac{1}{4}} V^\top$	6e-4	0	2.976
Second order	$U\Sigma^{\frac{1}{4}} V^\top$	6e-3	0	2.882
Second order	$U\Sigma^{\frac{1}{4}} V^\top$	1e-2	0	2.877
Second order	$U\Sigma^{\frac{1}{4}} V^\top$	2e-2	0	2.872
Second order	$U\Sigma^{\frac{1}{4}} V^\top$	3e-2	0	2.873
Second order	$U\Sigma^{\frac{1}{4}} V^\top$	4e-2	0	2.876
Second order	$U\Sigma^0 V^\top$	1e-3	0	2.921
Second order	$U\Sigma^0 V^\top$	3e-3	0	2.887
Second order	$U\Sigma^0 V^\top$	6e-3	0	2.880
Second order	$U\Sigma^0 V^\top$	7e-3	0	2.879
Second order	$U\Sigma^0 V^\top$	8e-3	0	2.880
Second order	$U\Sigma^0 V^\top$	1e-2	0	2.880